

Autenticación con JWT

4. Autenticación con JWT

Autenticación con JWT

Al terminar el apartado anterior, tus usuarios ya eran reales —persistidos en PostgreSQL, con la contraseña protegida por BCrypt—. Pero seguía quedando el problema señalado desde entonces: con HTTP Basic, cada petición volvía a mandar usuario y contraseña, aunque fuera solo codificados en Base64, no cifrados.

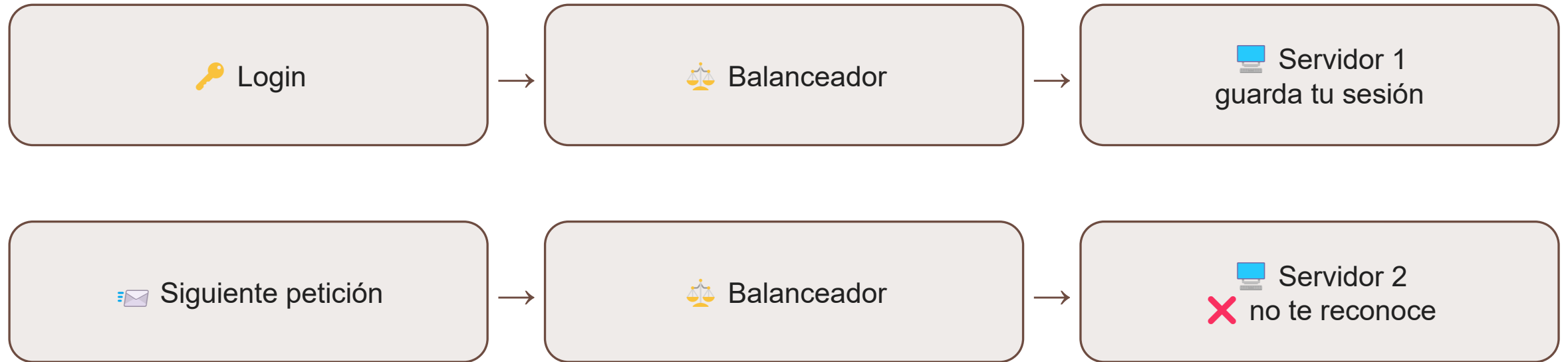
HTTP es un protocolo sin estado: cada petición llega sola, sin memoria de la anterior. El servidor no "recuerda" que hace un segundo te has autenticado. Hoy sustituyes eso por algo mejor.

Solución histórica 1: sesión en el servidor

El servidor, tras el login, guarda en su propia memoria (o en una base de datos de sesiones) quién eres, y te entrega una cookie con un identificador de sesión. En cada petición posterior, mandas esa cookie, y el servidor busca "¿quién es el dueño de esta sesión?".

Límite: el servidor tiene que guardar el estado de cada sesión activa. El problema aparece en cuanto tu aplicación corre en varias copias detrás de un balanceador de carga.

El balanceador rompe la sesión

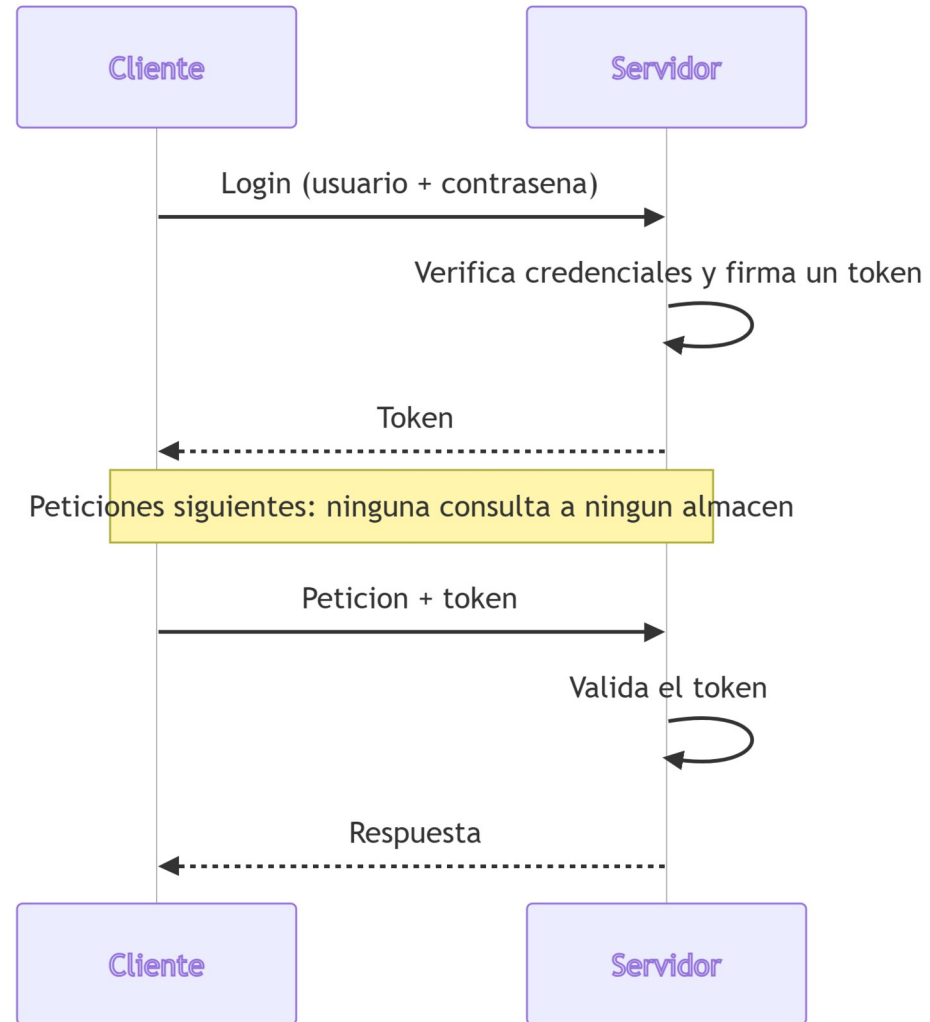


Tu sesión vive solo en la memoria del Servidor 1 — si la siguiente petición cae en el Servidor 2, es como si no hubieras iniciado sesión nunca.

Solución histórica 2: token autocontenido

El servidor no necesita guardar el estado de cada sesión. En el login, te entrega un token: un "carné" protegido criptográficamente que contiene, dentro de sí mismo, quién eres y qué puedes hacer. En cada petición posterior, presentas ese token, y el servidor puede validarlo sin consultar ningún almacén de sesiones.

El flujo del token autocontenido



Lo que cambia es la frecuencia

El login sigue mandando usuario y contraseña — eso no desaparece, y ahí sigues expuesto igual que con HTTP Basic. Lo que cambia es la frecuencia: en vez de en todas las peticiones, viajan una única vez, contra un único endpoint (/login) que puedes proteger a fondo.

JWT (JSON Web Token) es el formato estándar de ese token autocontenido, y es lo que vas a implementar hoy: resuelve el problema de la sesión en servidor porque no hay que compartir ni coordinar un almacén entre copias de la aplicación.

Qué significa "firmado"

Firmar no es lo mismo que cifrar. Proteger un dato mediante una firma o un código de autenticación no oculta su contenido: sigue siendo legible por cualquiera. Lo que permite es comprobar que no se ha modificado y que fue generado por alguien que conocía la clave necesaria.

Utilizarás HMAC (Hash-based Message Authentication Code) con el algoritmo HS256: combina una función hash —SHA-256— con un secreto compartido. Solo quien conoce ese secreto puede generar un código válido.

HS256: un código MAC simétrico

Aunque habitualmente se hable de «firmar un JWT», HS256 utiliza técnicamente un código MAC simétrico: la misma clave sirve para generar y para verificar el token. Su finalidad es comprobar la integridad y autenticidad del mensaje, no proteger contraseñas como hace BCrypt.

Anatomía de un JWT

El JWT tiene tres partes separadas por puntos: header.payload.signature, codificadas mediante Base64URL:

```
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhZG1pbGlzInJvbGVzIjpb...fQ.SflKxwRJ...
```

Parte	Qué contiene
header	Metadatos sobre la protección del token, como el algoritmo (alg).
payload	Los claims: los datos reales, quién eres y qué puedes hacer.
signature	El código HMAC calculado sobre header y payload, con el secreto del servidor.

Los claims del payload

```
{"sub": "admin", "roles": ["ADMIN"], "iat": 1730000000, "exp": 1730003600}
```

Campo	Qué es	¿De dónde sale?
sub	Quién eres — el usuario autenticado	Estándar de JWT
iat	Cuándo se emitió el token (timestamp Unix)	Estándar de JWT
exp	Cuándo caduca el token	Estándar de JWT
roles	Qué puedes hacer — tus roles	Personalizado: lo añades tú

⚠ Dos escenarios que conviene no mezclar

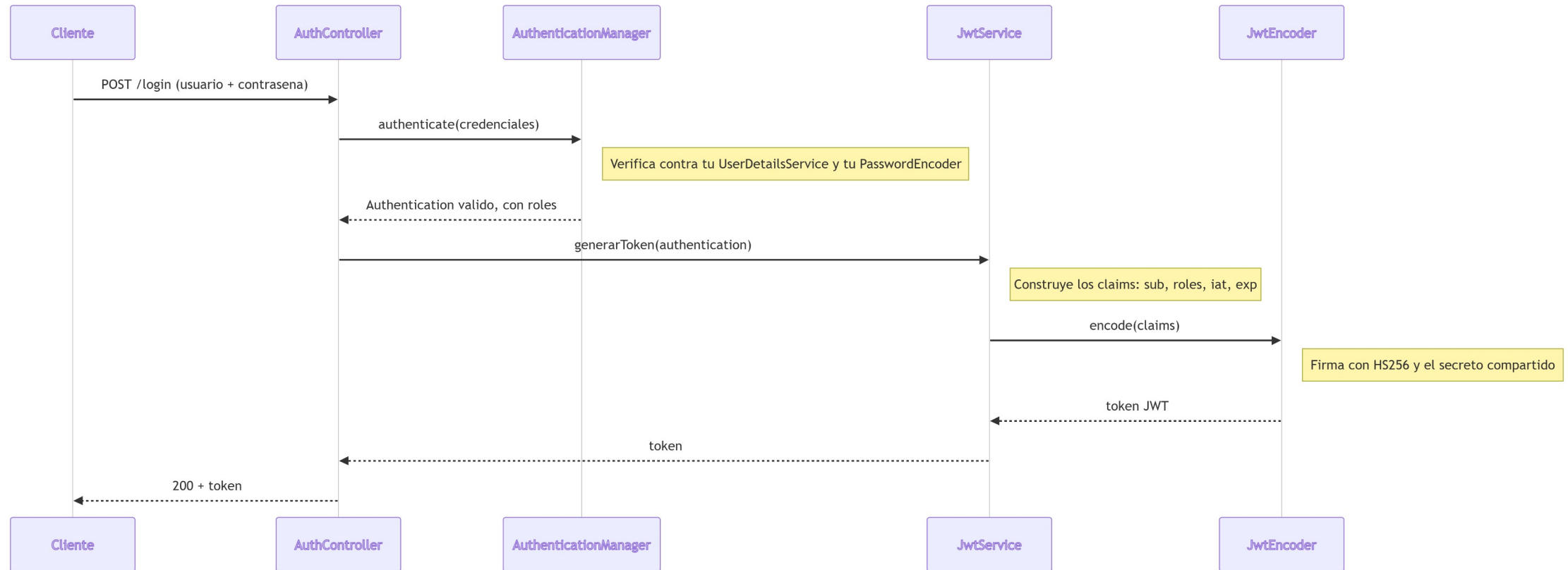
Si alguien roba un token válido y lo reenvía tal cual, el servidor lo acepta como credencial válida — la firma no protege de esto, por eso necesitas HTTPS: cifra el transporte para que nadie pueda leer u obtener el token.

Si alguien intenta modificarlo o fabricar uno nuevo —por ejemplo, cambiando tu rol de USER a ADMIN—, ahí sí entra la firma: se calcula con HMAC y un secreto que solo conoce el servidor. Sin ese secreto, es computacionalmente inviable calcular un código válido.

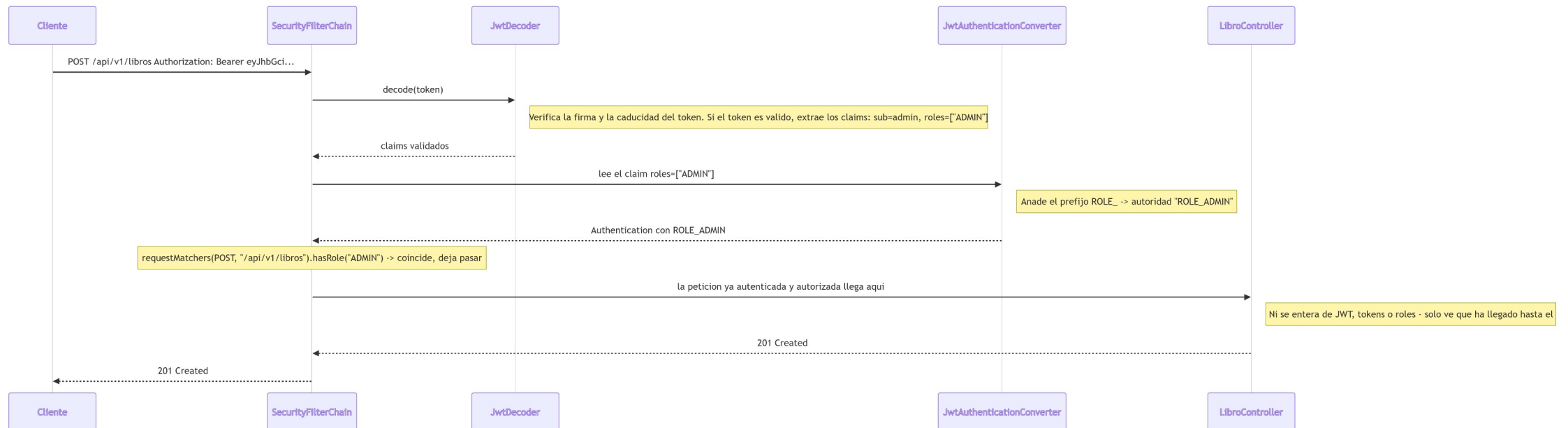
El flujo completo, pieza a pieza

Antes de entrar en el código de cada pieza, aquí tienes el mapa completo: primero cómo se genera el token en el login, después cómo se valida ese mismo token en cada petición posterior. Los nombres son los mismos que verás en el código — vuelve a este mapa si te pierdes.

Generando el token en el login



Validando el token en una petición posterior



SecurityFilterChain no genera la respuesta

SecurityFilterChain solo decide si la petición llega o no a tu controller. Si `hasRole("ADMIN")` coincide, la deja pasar sin más — tu `LibroController` ejecuta el mismo código que con HTTP Basic, sin saber nada de JWT—; si no coincide, la petición nunca llega, y es SecurityFilterChain quien responde con 403.

Si el token fuera de un usuario sin ADMIN, la firma seguiría siendo válida, pero la autoridad sería `ROLE_USER`: la firma certifica que el token es auténtico, no que tengas permiso para todo.

El secreto y los beans de SecurityConfig

JwtEncoder, JwtDecoder, JwtAuthenticationConverter... vienen de un starter que no habías necesitado hasta ahora:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
```

JwtEncoder firma tokens nuevos; JwtDecoder verifica los que llegan. Ambos necesitan la misma clave: el secreto con el que se calcula la firma HMAC — el mismo en los dos lados.

Los beans JwtEncoder y JwtDecoder

```
@Value("${libreria.jwt.secret}")
private String jwtSecret;

@Bean
public JwtEncoder jwtEncoder() {
    return new NimbusJwtEncoder(new ImmutableSecret<>(jwtSecret.getBytes(StandardCharsets.UTF_8)));
}

@Bean
public JwtDecoder jwtDecoder() {
    SecretKeySpec secretKey = new SecretKeySpec(jwtSecret.getBytes(StandardCharsets.UTF_8),
"HmacSHA256");
    return NimbusJwtDecoder.withSecretKey(secretKey).macAlgorithm(MacAlgorithm.HS256).build();
}
```

El secreto tiene que ser largo y aleatorio

⚠ HS256 requiere al menos 256 bits (32 bytes)

Como en este código conviertes directamente el texto a bytes UTF-8, una cadena ASCII de 32 caracteres ocupa esos 32 bytes mínimos. La longitud no basta por sí sola:

"aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa" tiene 32 caracteres, pero sigue siendo predecible. Utiliza una cadena generada aleatoriamente y guardada fuera del repositorio.

Una clave demasiado corta será rechazada cuando la biblioteca intente utilizarla para firmar o validar el token.

AuthenticationManager: qué hace por debajo

Recibe unas credenciales todavía sin comprobar y, si son correctas, devuelve un Authentication ya validado. Por debajo usa dos piezas que ya tienes en tu contexto:



Exponerlo como bean inyectable

Spring Security construye internamente el `AuthenticationManager`, pero no lo publica como bean por su cuenta. `AuthenticationConfiguration.getAuthenticationManager()` es la forma soportada de pedírsela:

```
@Bean
public AuthenticationManager authenticationManager(AuthenticationConfiguration configuration)
    throws Exception {
    return configuration.getAuthenticationManager();
}
```

Generar el token: JwtService

```
@Service
@RequiredArgsConstructor
public class JwtService {

    private final JwtEncoder jwtEncoder;

    @Value("${libreria.jwt.expiration-minutes}")
    private long expirationMinutes;

    public String generarToken(Authentication authentication) {
        Instant ahora = Instant.now();
        List<String> roles = authentication.getAuthorities().stream()
            .map(GrantedAuthority::getAuthority)
            .filter(a -> a.startsWith("ROLE_"))
            .map(a -> a.replace("ROLE_", ""))
            .toList();

        JwtClaimsSet claims = JwtClaimsSet.builder()
            .issuer("libreria").issuedAt(ahora)
            .expiresAt(ahora.plusSeconds(expirationMinutes * 60))
            .subject(authentication.getName()).claim("roles", roles).build();

        JwsHeader jwsHeader = JwsHeader.with(MacAlgorithm.HS256).build();
        return jwtEncoder.encode(JwtEncoderParameters.from(jwsHeader, claims)).getTokenValue();
    }
}
```

Qué construye JwtService

Ese Authentication es el mismo del diagrama de antes: te lo entrega `authenticationManager.authenticate(...)`. Los claims son los datos que viajan dentro del token: `subject` (quién eres), `roles` (qué puedes hacer), `issuedAt/expiresAt` (cuándo se emitió y caduca). `jwtEncoder.encode(...)` firma todo con HS256 y el secreto configurado.

`getExpiresInSeconds()` expresa esa duración en segundos, para que el cliente sepa cuánto le queda de validez sin decodificar el token.

El endpoint de login: DTOs

```
public record LoginRequestDTO(  
    @NotBlank(message = "El nombre de usuario no puede estar vacío") String username,  
    @NotBlank(message = "La contraseña no puede estar vacía") String password  
) {}  
  
public record LoginResponseDTO(String accessToken, String tokenType, long expiresInSeconds) {}
```

LoginResponseDTO no llama a su campo token a secas, sino accessToken; tokenType viaja siempre como "Bearer", el prefijo que el cliente antepondrá en la cabecera Authorization.

AuthController.login(...)

```
@PostMapping("/login")
public ResponseEntity<LoginResponseDTO> login(@Valid @RequestBody LoginRequestDTO dto) {
    Authentication authentication = authenticationManager.authenticate(
        new UsernamePasswordAuthenticationToken(dto.username(), dto.password())
    );
    String token = jwtService.generarToken(authentication);
    return ResponseEntity.ok(new LoginResponseDTO(token, "Bearer",
        jwtService.getExpiresInSeconds()));
}
```

`authenticationManager.authenticate(...)` dispara la verificación (tu `UserDetailsService` + tu `PasswordEncoder`). Si son correctas, devuelve el `Authentication`; si no, lanza una `AuthenticationException`.

Una diferencia importante con el 401 de HTTP Basic

El 401 de HTTP Basic nunca llegaba a tu `GlobalExceptionHandler` —lo generaba un filtro, antes del `DispatcherServlet`—. Esta excepción es distinta: `authenticate(...)` se llama dentro de tu propio método de controller, no en ningún filtro, así que sí pasa por el ciclo normal de Spring MVC.

Un sexto @ExceptionHandler

```
@ExceptionHandler(AuthenticationException.class)
public ResponseEntity<ErrorResponse> handleAuthenticationException(
    AuthenticationException ex, HttpServletRequest request) {

    ErrorResponse response = new ErrorResponse(
        LocalDateTime.now().toString(), 401, "No autenticado",
        "Usuario o contraseña incorrectos", request.getRequestURI());
    return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(response);
}
```

Añádalo a la misma clase de siempre, junto a los otros cinco. Es un caso distinto al `AuthenticationEntryPoint` de la Actividad 2.2, aunque los dos acaben resolviendo un 401.

AuthenticationEntryPoint vs. este @ExceptionHandler

	AuthenticationEntryPoint (2.2)	Este @ExceptionHandler (hoy)
¿Quién genera el fallo?	Spring Security, en un filtro, antes de cualquier controller	Tu propio código, dentro de AuthController.login(...)
¿Llega a un controller?	No — se corta antes	Sí — la ruta es permitAll(), y falla ya dentro
¿Pasa por DispatcherServlet?	No	Sí

El cambio de modo en SecurityConfig

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http,
    AuthenticationEntryPoint customAuthenticationEntryPoint) throws Exception {
    return http
        .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(POST, "/api/v1/auth/register", "/api/v1/auth/login").permitAll()
            .requestMatchers(GET, "/api/v1/libros/**").permitAll()
            .requestMatchers("/v3/api-docs/**", "/swagger-ui/**", "/documentacion").permitAll()
            .anyRequest().authenticated()
        )
        .exceptionHandling(ex -> ex.authenticationEntryPoint(customAuthenticationEntryPoint))
        .csrf(AbstractHttpConfigurer::disable)
        .oauth2ResourceServer(oauth2 -> oauth2.jwt(jwt ->
            jwt.jwtAuthenticationConverter(jwtAuthenticationConverter())))
        .httpBasic(AbstractHttpConfigurer::disable)
        .build();
}
```

STATELESS, oauth2ResourceServer, httpBasic disable

- `SessionCreationPolicy.STATELESS`: consecuencia directa de usar tokens autocontenidos — con JWT no hace falta que el servidor guarde ninguna sesión.
- `oauth2ResourceServer(oauth2 -> oauth2.jwt(...))` activa la autenticación mediante JWT en las peticiones con token Bearer, usando el `JwtDecoder` configurado.
- `httpBasic(AbstractHttpConfigurer::disable)` retira oficialmente el mecanismo provisional: JWT es ahora el único mecanismo de autenticación.
- La regla de `/v3/api-docs/**`, `/swagger-ui/**` y `/documentacion` es nueva: sin ella, el navegador no podría ni cargar la documentación bajo STATELESS.

JwtAuthenticationConverter: tus roles, no scope

Por defecto, Spring espera los roles en un claim scope o scp — el habitual en OAuth2. Tú los guardas en un claim propio, roles:

```
@Bean
public JwtAuthenticationConverter jwtAuthenticationConverter() {
    JwtGrantedAuthoritiesConverter authoritiesConverter = new JwtGrantedAuthoritiesConverter();
    authoritiesConverter.setAuthoritiesClaimName("roles");
    authoritiesConverter.setAuthorityPrefix("ROLE_");

    JwtAuthenticationConverter authenticationConverter = new JwtAuthenticationConverter();
    authenticationConverter.setJwtGrantedAuthoritiesConverter(authoritiesConverter);
    return authenticationConverter;
}
```

`setAuthorityPrefix("ROLE_")` reconstruye el prefijo que `JwtService` te quitó al generar el token, para no guardarlo por duplicado dentro del JWT.

GET /api/v1/auth/me

```
@GetMapping("/me")
public ResponseEntity<AuthMeResponse> getCurrentUser() {
    Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
    // construir la respuesta con authentication.getName() y sus roles
}
```

Un endpoint sencillo para verificar qué información viaja dentro de tu propio token: útil tanto para probar como para entender qué sabe el servidor de ti en cada petición autenticada.

Configurar "Authorize" en Swagger para Bearer

```
@Bean
public OpenAPI libreriaOpenAPI() {
    final String esquema = "bearerAuth";
    return new OpenAPI()
        .info(new Info().title("Mi librería").version("v1"))
        .components(new Components()
            .addSecuritySchemes(esquema, new SecurityScheme()
                .type(SecurityScheme.Type.HTTP)
                .scheme("bearer").bearerFormat("JWT")))
        .addSecurityItem(new SecurityRequirement().addList(esquema));
}
```

`addSecuritySchemes(...)` define el esquema solo para que Swagger sepa cómo mostrarlo, no cambia nada del backend;
`addSecurityItem(...)` lo aplica por defecto a todos los endpoints.

Cómo se prueba en Swagger UI

- Reinicia, entra en /documentacion, busca POST /api/v1/auth/login y pruébalo con "Try it out" — sin pulsar "Authorize" antes, no lo necesita.
- Copia el accessToken de la respuesta.
- Pulsa "Authorize", pega el token (sin escribir Bearer, Swagger lo añade solo) y prueba cualquier endpoint protegido con "Try it out".

HTTPS: lo que JWT resuelve y lo que no

JWT evita reenviar la contraseña en cada petición — un avance real. Pero el canal en sí sigue siendo el mismo: si no hay HTTPS, tanto el login como el propio token viajan en claro por la red. La firma del JWT garantiza integridad, no confidencialidad del transporte — son dos cosas distintas.

HTTPS añade esa confidencialidad cifrando el canal con TLS. En desarrollo se suele usar un certificado autofirmado; en producción lo emite una CA reconocida, o se delega a un proxy/gateway. El resto de este curso sigue trabajando sin HTTPS, por simplificación deliberada.

Lo que JWT tampoco resuelve: revocar un token

Si un usuario cierra sesión, o un administrador quiere echarlo ahora mismo, ¿cómo se invalida su token? Con JWT tal y como lo has construido, no se puede: el servidor no guarda ningún registro de qué tokens ha emitido, así que un token firmado sigue siendo válido hasta que caduca por su cuenta (exp).

Marcar un usuario como activo = false impide que vuelva a autenticarse, pero no invalida un JWT que ya tenía emitido. En un proyecto real: lista negra de tokens revocados, o tokens de vida muy corta + refresh token.

Ideas clave

- HTTP es sin estado; la sesión en servidor (cookie + almacén) y el token autocontenido (JWT) son las dos soluciones clásicas para "recordar quién eres".
- Proteger $\neq$ cifrar: el contenido de un JWT es legible, pero HMAC permite detectar modificaciones y comprobar que fue generado con el secreto compartido.
- El JWT tiene tres partes (header.payload.signature); los claims del payload llevan quién eres, tus roles y la expiración.
- `SessionCreationPolicy.STATELESS` + `oauth2ResourceServer(...).jwt(...)` activan la validación automática de JWT; `httpBasic(disable)` retira el mecanismo provisional.

Ideas clave (cont.)

- Spring construye internamente el `AuthenticationManager`, pero hay que exponerlo como bean para inyectarlo; el `JwtAuthenticationConverter` es propio porque tus roles están en el claim roles, no en scope/scp.
- JWT resuelve el reenvío de contraseña en cada petición, pero no sustituye a HTTPS — la firma da integridad, no confidencialidad del canal.
- `authenticate(...)` se llama dentro de tu controller, no en un filtro — su excepción sí llega a tu `GlobalExceptionHandler`, a diferencia del 401 de HTTP Basic.
- JWT tampoco resuelve la revocación: un token firmado es válido hasta que caduca — se aborda con listas negras o tokens de vida corta + refresh token, ninguna implementada en este curso.